

Hardware/Software Co-design: Application Specific Processors

System Level Design and Modeling

Preeti Ranjan Panda

Department of Computer Science and Engineering

Indian Institute of Technology Delhi

Adding a new instruction

- Example new function: fft, image compression, exp(x), trig function
 - exp(x): instruction format – Opcode, Dest, Source
 - sin(x): same instruction format – Opcode, Dest, Source

Hardware Software Partitioning

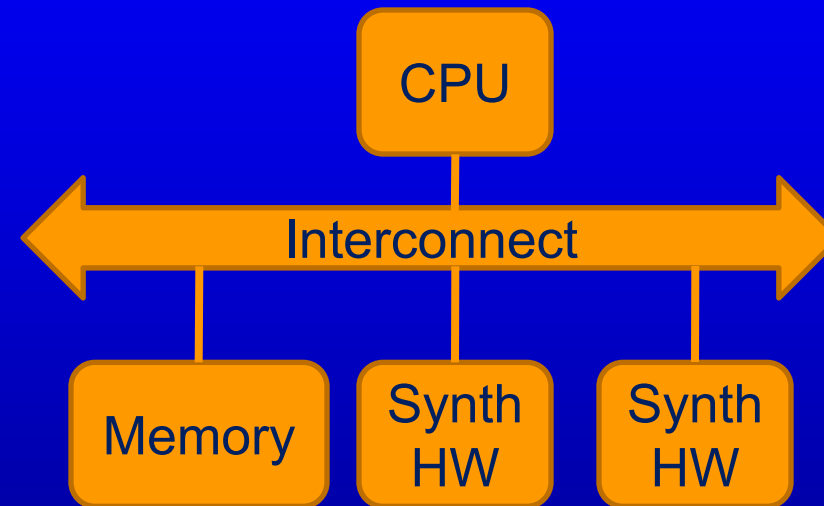
- Some blocks mapped to HW
- Other blocks mapped to SW
- Evaluation metrics
 - Area
 - Performance
 - Energy/power
 - Energy x Delay
 - Energy x Delay²

Granularity of mapping

- Module
- Processes
- Functions, loops,...

What is a Reasonable Architecture?

- Blocks connected over an interconnect architecture
 - CPU cores
 - Memory (cache, scratch pad)
 - Application specific HW



Other Architectures

- How about modifying the CPU itself?
 - Advantage: can control data movement more accurately
 - Disadvantage: need to repeat design flow
 - re-synthesis, verification, physical design

Processors in Embedded Systems

- General purpose processors
- Special purpose off the shelf processors
 - DSPs
 - Micro controllers
 - GPUs
- Processors customized for the application:
 - Application Specific Instruction-set Processors (ASIPs)
 - Exploit special characteristics to meet the desired constraints

Slides 6-25 source: Anshul Kumar, "Application Specific Instruction Processors"

Architecture Specialization in Embedded Systems

- System specialization
- Component specialization

System specialization

- Partition the application and map it to components
 - Control decomposition
 - Data Decomposition
- Component interaction

Component Specialization Techniques

- Instruction set specialization
- Data path specialization
- Memory specialization
- Interconnect specialization

Instruction set specialization

- Define operations that can be done in parallel - instruction level parallelism
 - e.g., address computation and arithmetic
- Operations that can be chained
 - classical example - multiply accumulate
- Create frequently occurring special cases of general instructions
 - Reduces instruction size, time, power

Data path specialization

- Word length adaptation
- No. of Function Units (FUs)
- Application specific FUs
- Register file size/structure

Memory specialization

- No. and size of banks
- No. and size of access ports
- Access patterns (page mode, interleaving)
- Cache organization (size, block size, levels, policies, etc.)
- Special memories - scratch pads, FIFOs